Chapter 06: Spring Data JPA Basic

Department of Software Engineering
Faculty of Information Technology (HCM-UTE)
Course: Web Programming
Course ID: WEPR330479
Lecturer: Msc. Khoa Hoang

Content

- Spring Data JPA
- Entity mapping
- Spring repository
- Transaction

Spring Data JPA

Bridging object-oriented Java and relational databases

Overview

- Spring Data JPA is part of the Spring Framework ecosystem and simplifies interactions with relational databases.

- Official documentation: https://spring.io/projects/spring-data-jpa

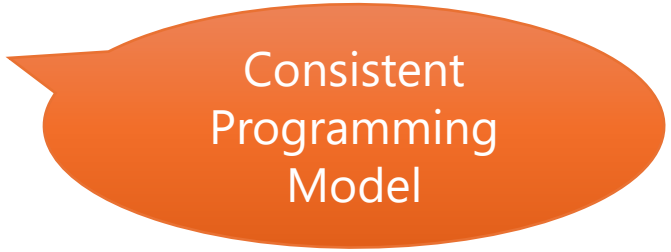
- Spring Data JPA serves as an abstraction layer over standard JPA and ORM implementations (such as Hibernate), helping to reduce boilerplate code and automate data query operations.

Spring Data project

- **Spring Data**
  - Spring Data Commons
  - Spring Data JPA
  - Spring Data KeyValue
  - Spring Data LDAP
  - Spring Data MongoDB
  - Spring Data Redis
  - Spring Data Relational
  - Spring Data REST
  - Spring Data for Apache Cassandra
  - Spring Data Couchbase
  - Spring Data Elasticsearch
  - Spring Data Envers
  - Spring Data Neo4j
  - Spring Data R2DBC

Main modules

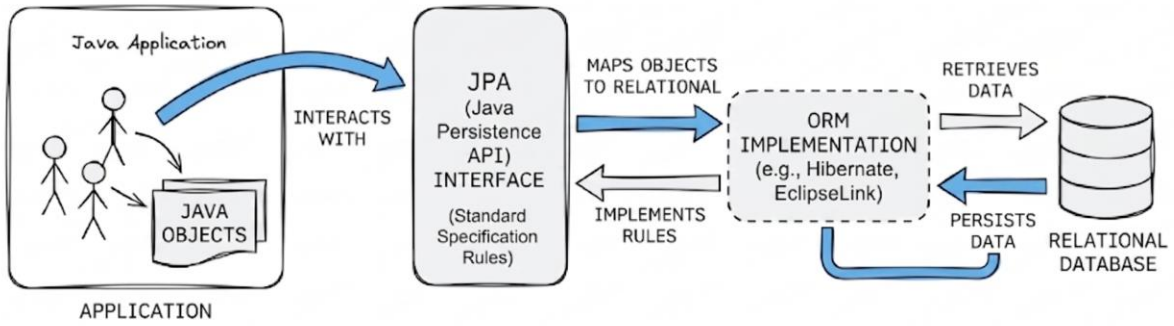
- Spring Data Commons - Core Spring concepts underpinning every Spring Data module.
- Spring Data Relational - Spring Data repository support for JDBC and R2DBC.
- Spring Data JPA - Spring Data repository support for JPA.
- Spring Data KeyValue - `Map` based repositories and SPIs to easily build a Spring Data module for key-value stores.
- Spring Data LDAP - Spring Data repository support for Spring LDAP.
- Spring Data MongoDB - Spring based, object-document support and repositories for MongoDB.
- Spring Data Redis - Easy configuration and access to Redis from Spring applications.
- Spring Data REST - Exports Spring Data repositories as hypermedia-driven RESTful resources.
- Spring Data for Apache Cassandra - Easy configuration and access to Apache Cassandra or large scale, highly available, data oriented Spring applications.



https://spring.io/projects/spring-data

JPA

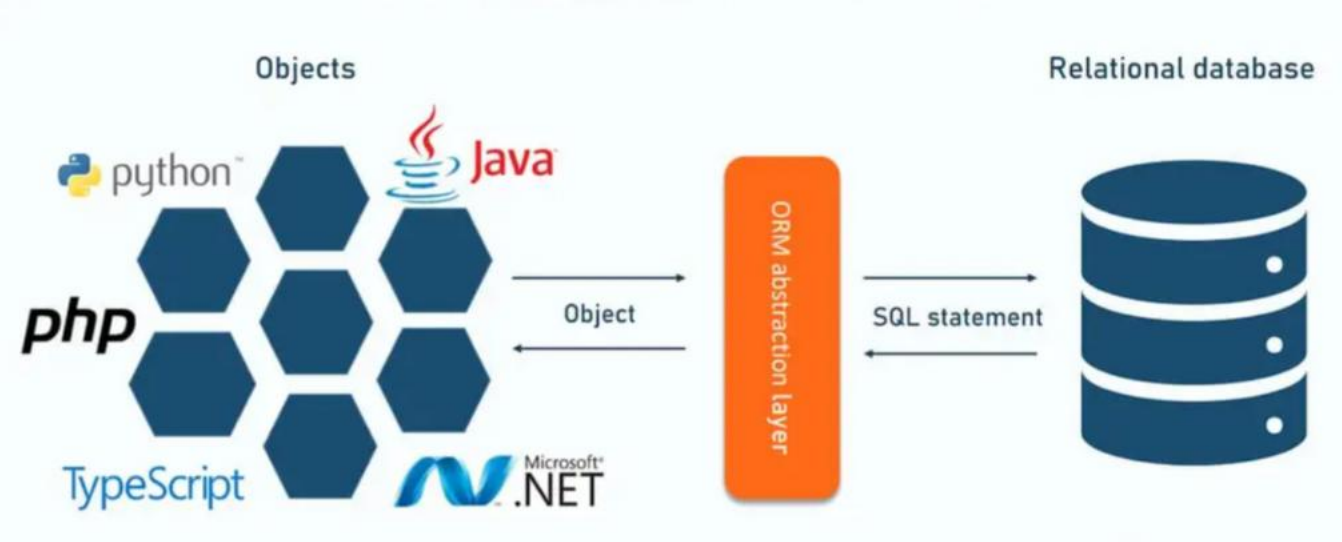
- JPA stands for Java/Jakarta Persistence API
- JPA is a standard Java **specification** defines the interface, rules and guidelines for Object-Relational Mapping (ORM)
- Official latest specification documentation: https://jakarta.ee/specifications/persistence/3.2/



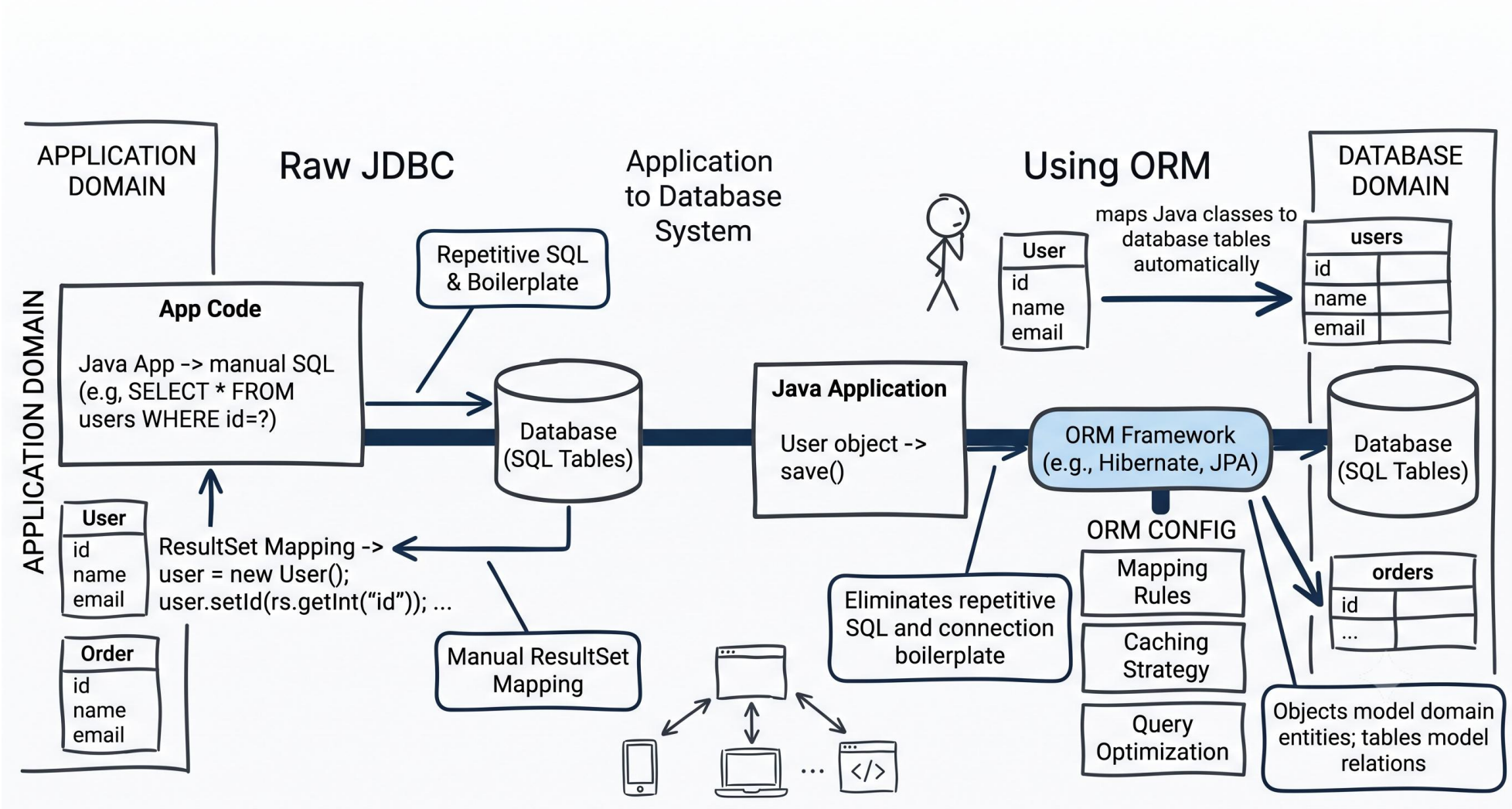
ORM

- ORM stands for Object-Relational Mapping

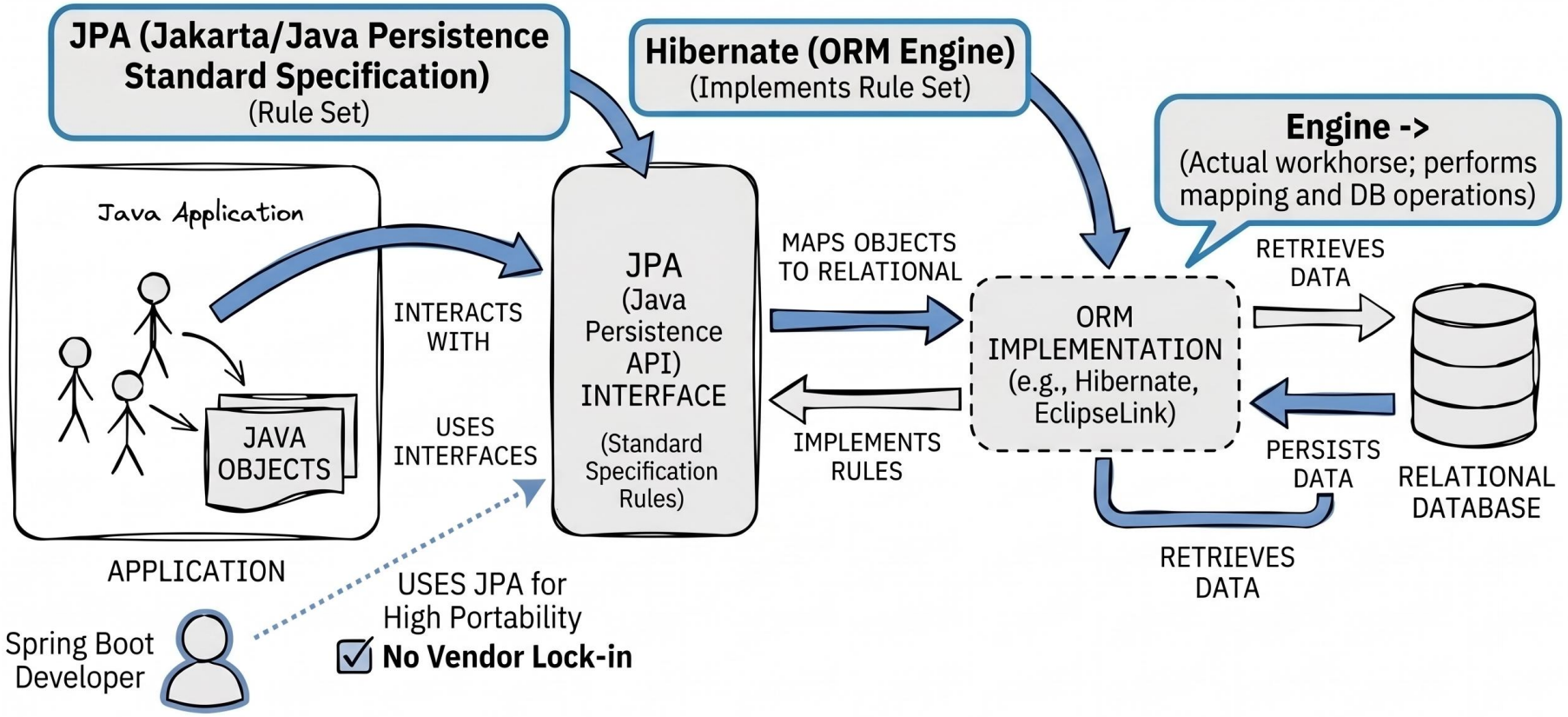
- ORM is a software technique that lets developers interact with a relational database using an object-oriented programming language, instead of writing raw SQL queries (**JPA's implementation**).



Why use Object-Relational Mapping?

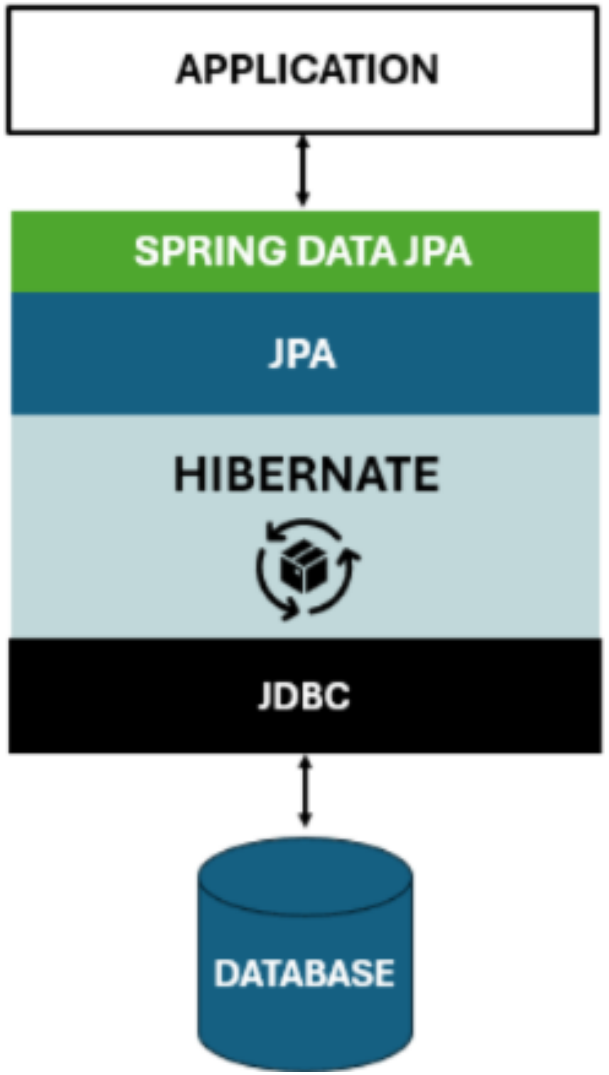


JPA and Hibernate ORM



- JPA is the standard interface rules for ORM.
- `Hibernate` acts as the concrete engine that maps data.
- `Spring Boot` uses the portable JPA interfaces, avoiding dependency on raw `Hibernate` APIs.

Spring Data JPA



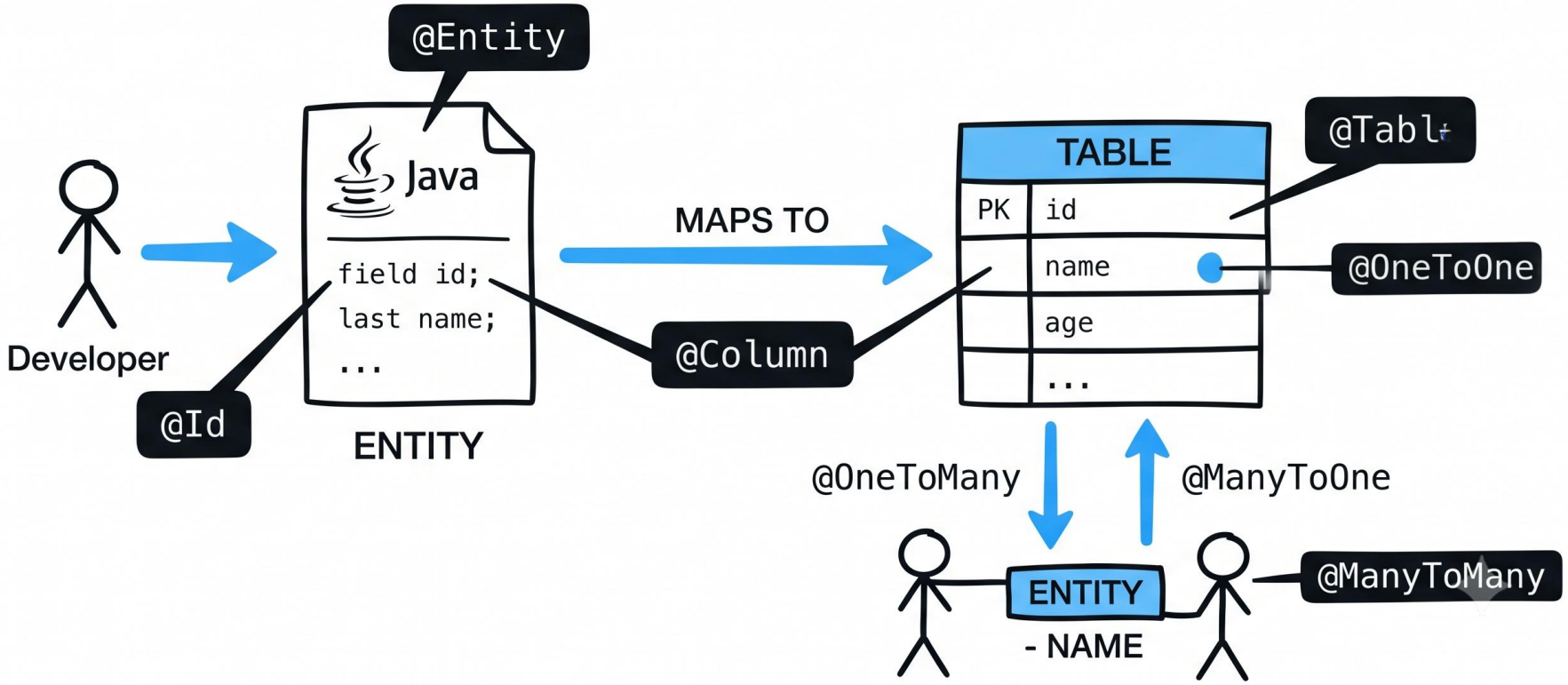
Entity mapping

Defining domain models with JPA annotations

Overview

- Entity mapping is at the core of the Java/Jakarta Persistence API (JPA)
- Allows developers to map Java objects (entities) to database tables
- The most common annotations:
  - Entities: `@Entity`, `@Table`, `@Id`, `@Column`, `@GeneratedValue`
  - Relationships: `@OneToOne`, `@OneToMany`, `@ManyToOne`, `@ManyToMany`

Entity mapping process

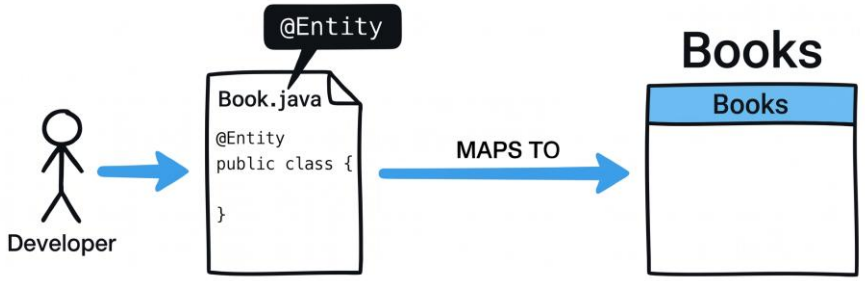


@Entity

- The @Entity annotation is used to mark a class as a JPA entity.

- This tells JPA that this class should be persisted in the database.

```java
@Entity
public class Book {
    // Fields and methods
}
```



@Id

- The @Id annotation marks a field as the primary key for the entity.
- The @GeneratedValue annotation specifies that it will be auto-generated.

```
@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String author;
}
```

@Column

- The @Column annotation is used to specify the mapping between a field and a database column.

```java
@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "book_title", length = 100, nullable = false)
    private String title;

    @Column(nullable = false)
    private String author;
}
```



@Table

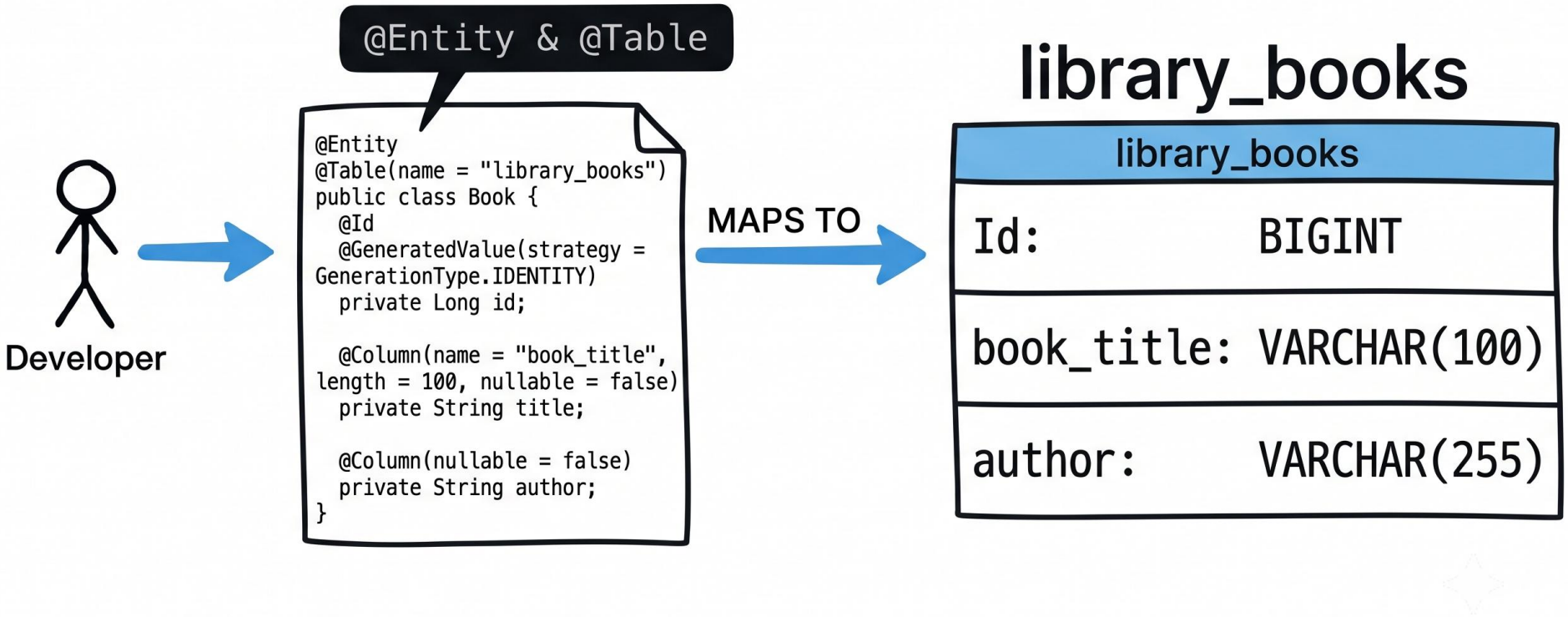
- The @Table annotation is used to specify the table name for the entity.

```java
@Entity
@Table(name = "library_books")
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(nullable = false)
    private String author;
}
```

Entity final mapping

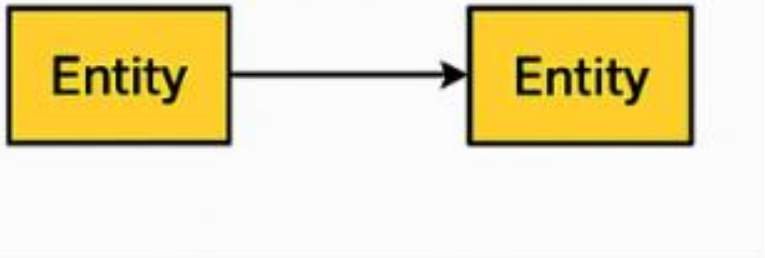


Relationship mapping: @OneToOne

- A @OneToOne relationship maps one entity to another entity.
- This is used when an entity has a single related entity.

```java
@Entity
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToOne
    private Address address;
}
```



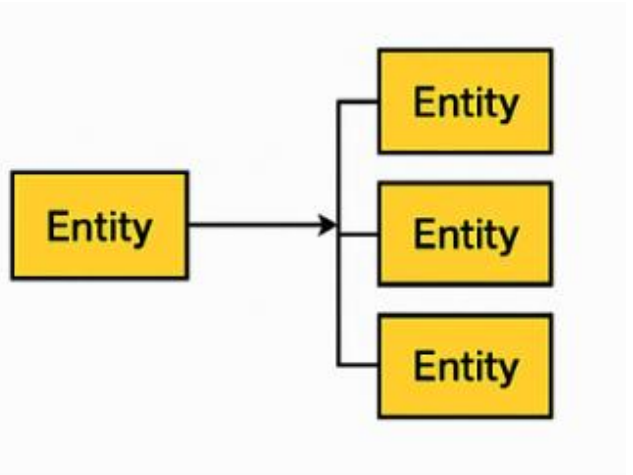
Relationship mapping: @OneToMany

- A @OneToMany relationship indicates that one entity can be associated with multiple related entities.
- The ***mappedBy*** attribute tells JPA that the author field in the Book entity manages the relationship.

```java
@Entity
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL)
    private List<Book> books;
}
```

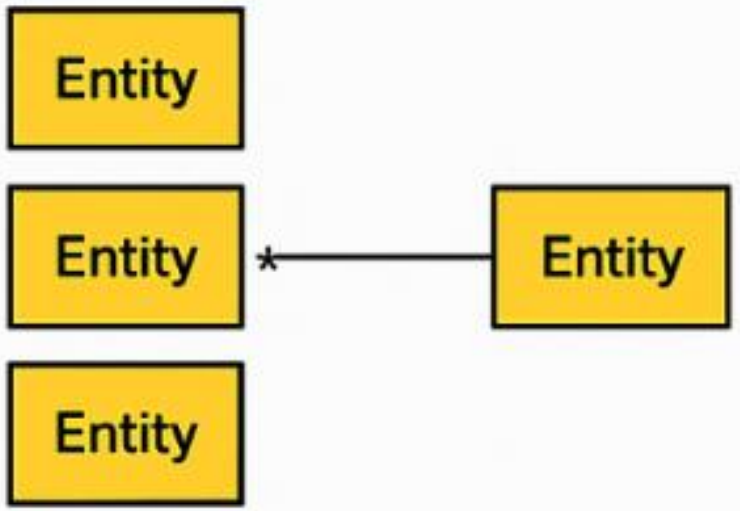


Relationship mapping: @ManyToOne

- A @ManyToOne relationship is the inverse of @OneToMany

```java
@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    private Author author;
}
```

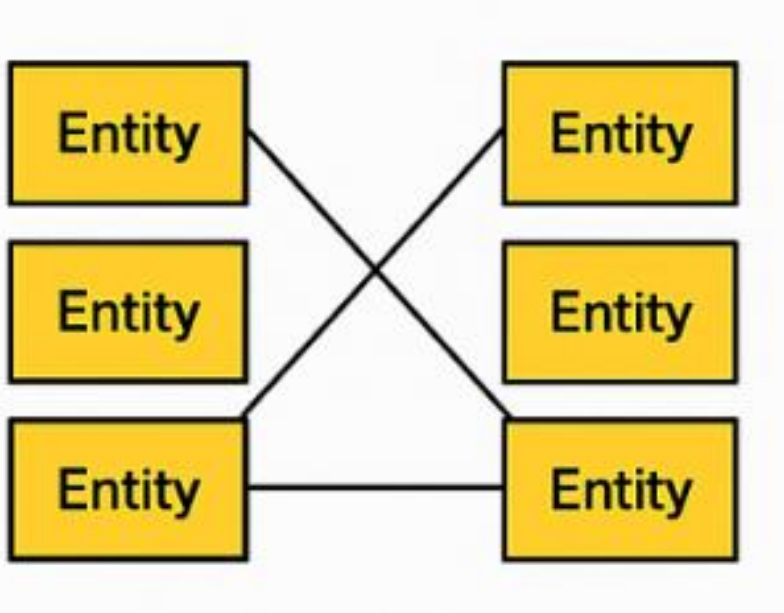


Relationship mapping: @ManyToMany #1

- A @ManyToMany relationship represents a relationship where multiple entities can be associated with multiple other entities.

```java
@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToMany
    @JoinTable(
      name = "book_category",
      joinColumns = @JoinColumn(name = "book_id"),
      inverseJoinColumns = @JoinColumn(name = "category_id"))
    private Set<Category> categories;
}
```



Relationship mapping: @ManyToMany #2

```java
@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToMany
    @JoinTable(
      name = "book_category",
      joinColumns = @JoinColumn(name = "book_id"),
      inverseJoinColumns = @JoinColumn(name = "category_id"))
    private Set<Category> categories;
}
```

```
CREATE TABLE books (
    id BIGINT PRIMARY KEY AUTO_INCREMENT
    -- other columns...
);

CREATE TABLE categories (
    id BIGINT PRIMARY KEY AUTO_INCREMENT
    -- other columns...
);

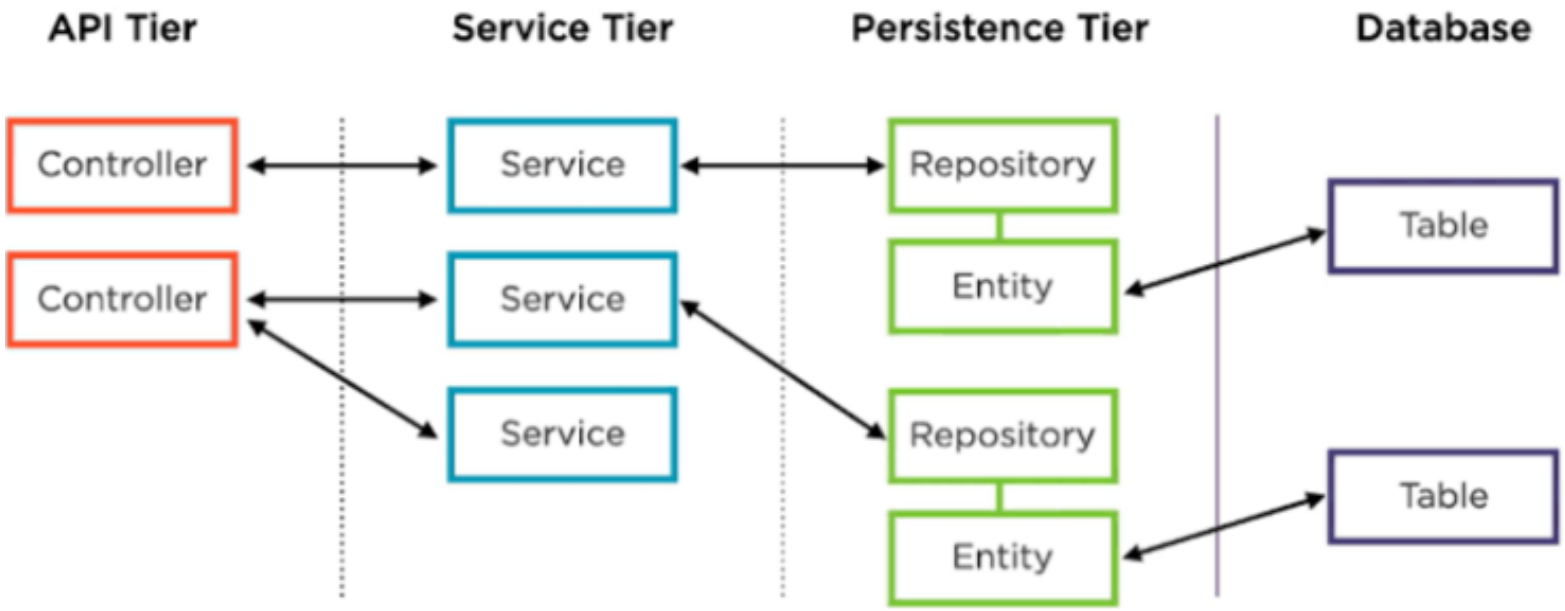
CREATE TABLE book_category (
    book_id BIGINT NOT NULL,
    category_id BIGINT NOT NULL,
    PRIMARY KEY (book_id, category_id),
    FOREIGN KEY (book_id) REFERENCES books(id),
    FOREIGN KEY (category_id) REFERENCES categories(id)
);
```

Spring repository

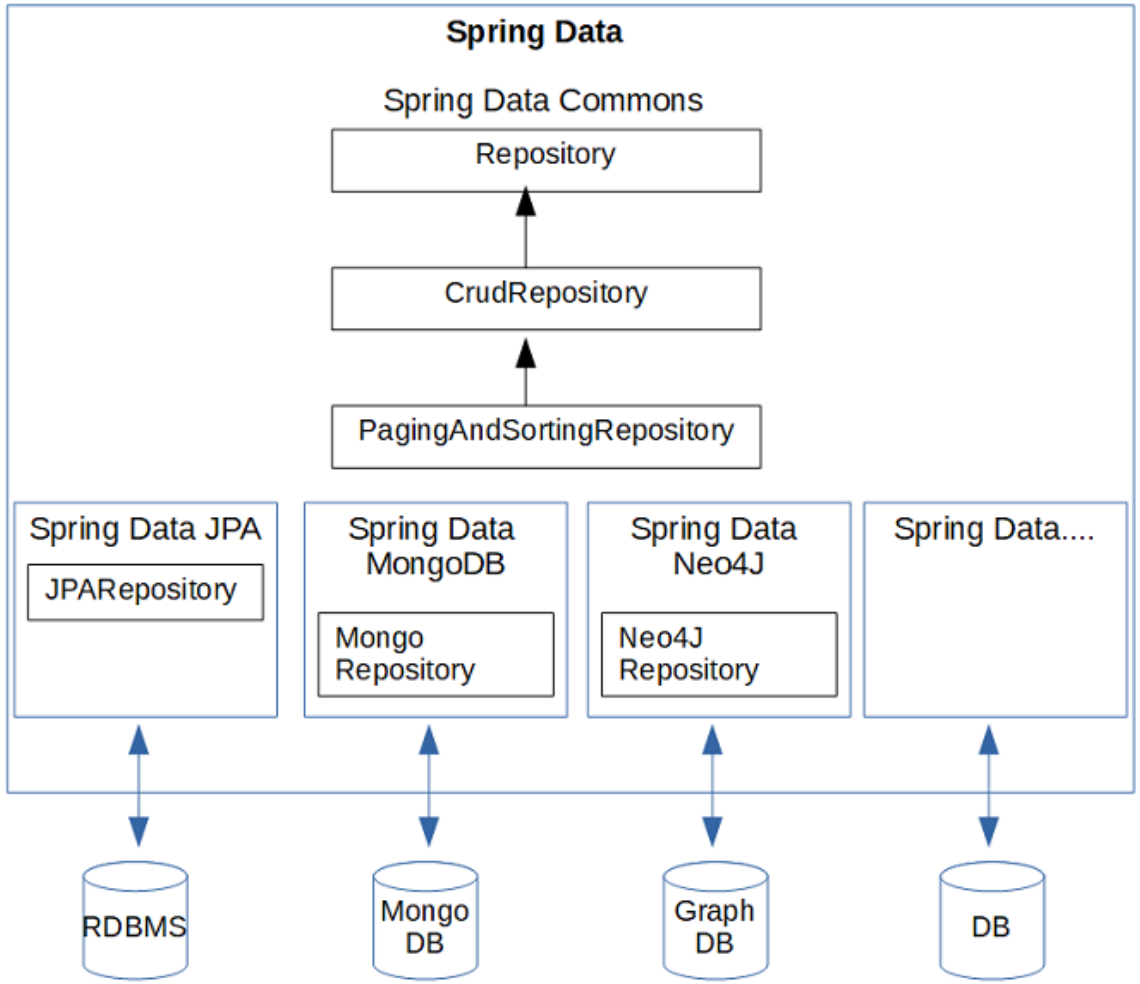
Repository abstraction without boilerplate queries

Repository

- A repository is an abstraction that simulates an in-memory collection of domain objects, hiding database access details.
- In other words, business logic shouldn't know whether it is interacting with a database or an in-memory list.



Spring repository hierarchy



https://docs.spring.io/spring-data/jpa/reference/repositories/core-concepts.html

JpaRepository interface #1

- JpaRepository is a key interface in Spring Data JPA that provides standard CRUD, sorting, and pagination methods for a domain entity, eliminating the need to write boilerplate data access code.
- It takes two generic parameters: the entity type and its primary key type (e.g., JpaRepository<User, Long>)

```
public interface UserRepository extends JpaRepository<User, Long> {

  List<User> findByLastname(String lastname);

  User findByEmailAddress(String emailAddress);
}
```

JpaRepository interface #2

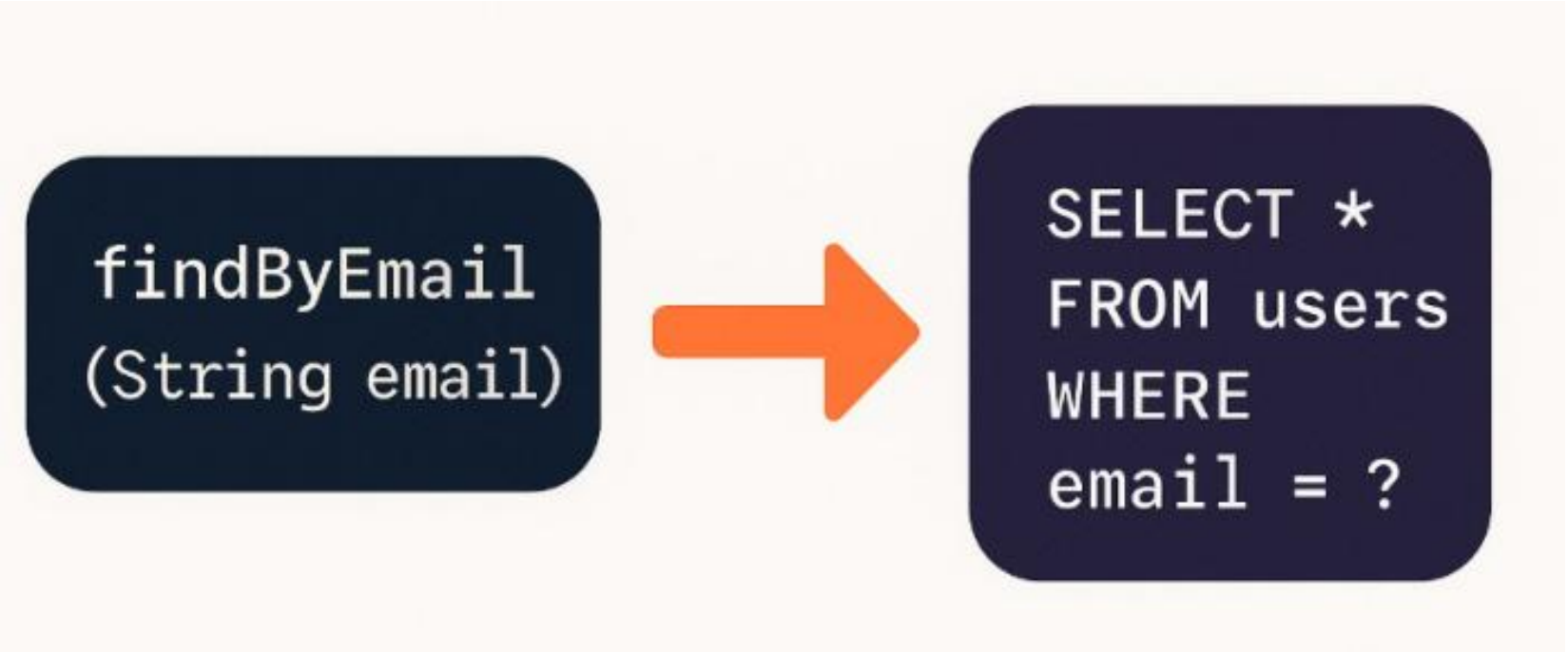
- Thanks to inheritance, UserRepository has over 20 methods available as followings

```
userRepository.save(user);
userRepository.findById(1L);
userRepository.findAll();
userRepository.findAll(PageRequest.of(0, 10));
userRepository.deleteById(1L);
userRepository.count();
userRepository.existsById(1L);
userRepository.flush();
// ... and many other methods
```

Without writing a single line of code, Spring automatically generates the implementation at **runtime**.

JpaRepository interface #3

- Spring Data JPA **Query Derivation** mechanism is a "convention-over-configuration" feature that automatically interprets your repository method signatures and builds corresponding Java Persistence Query Language (JPQL) queries at application startup



JpaRepository interface #4

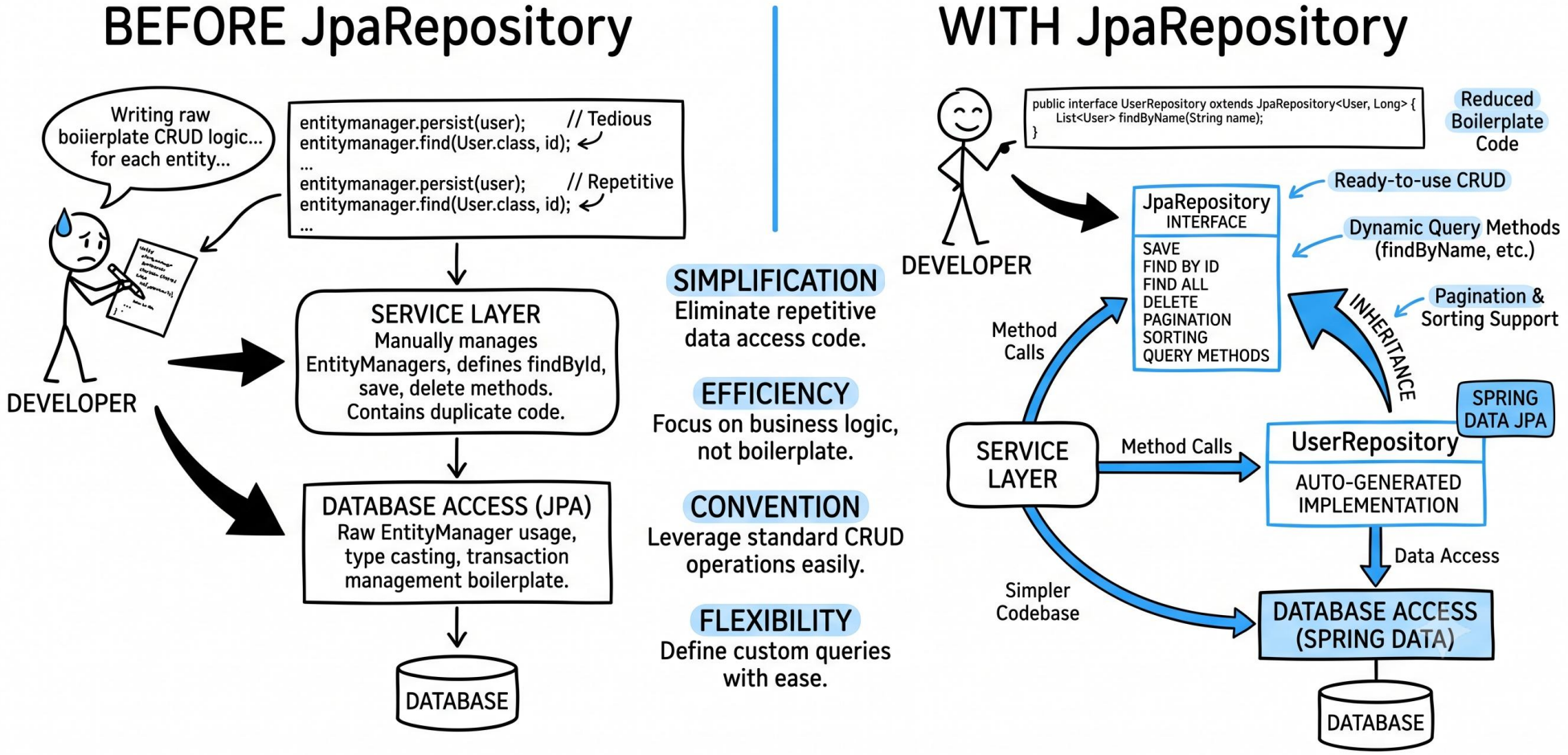
```
public interface UserRepository extends JpaRepository<User, Long> {

  @Query("select u from User u where u.emailAddress = ?1")
  User findByEmailAddress(String emailAddress);
}
```

```
public interface UserRepository extends JpaRepository<User, Long> {

  @Query("select u from User u where u.firstname like %?1")
  List<User> findByFirstnameEndsWith(String firstname);
}
```

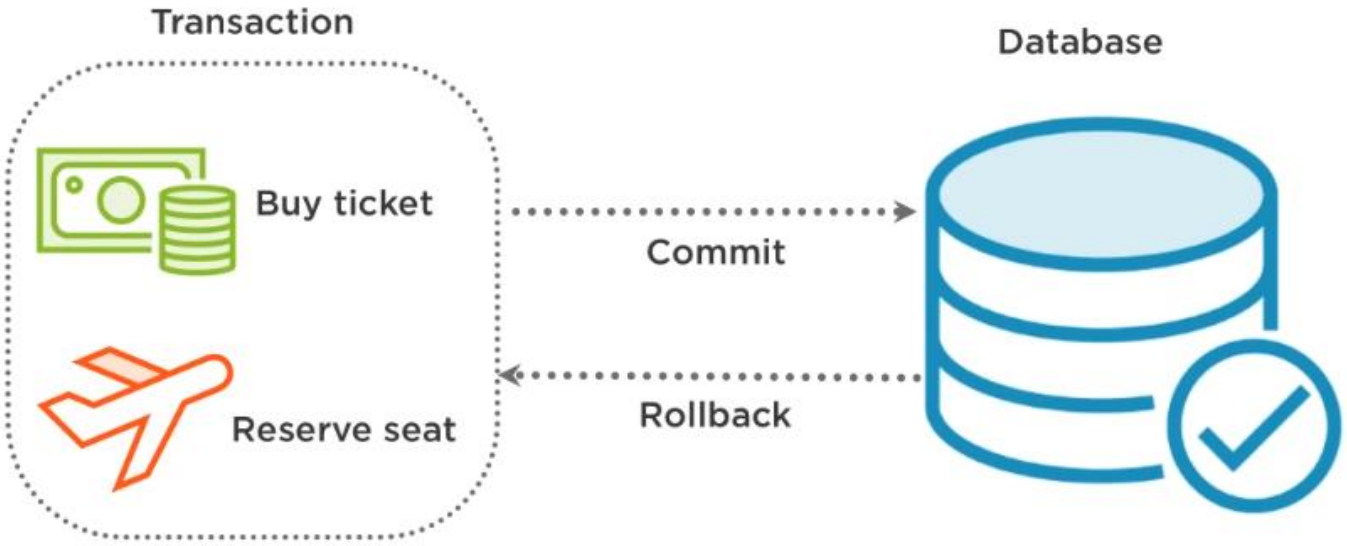
JpaRepository interface #5



Transaction

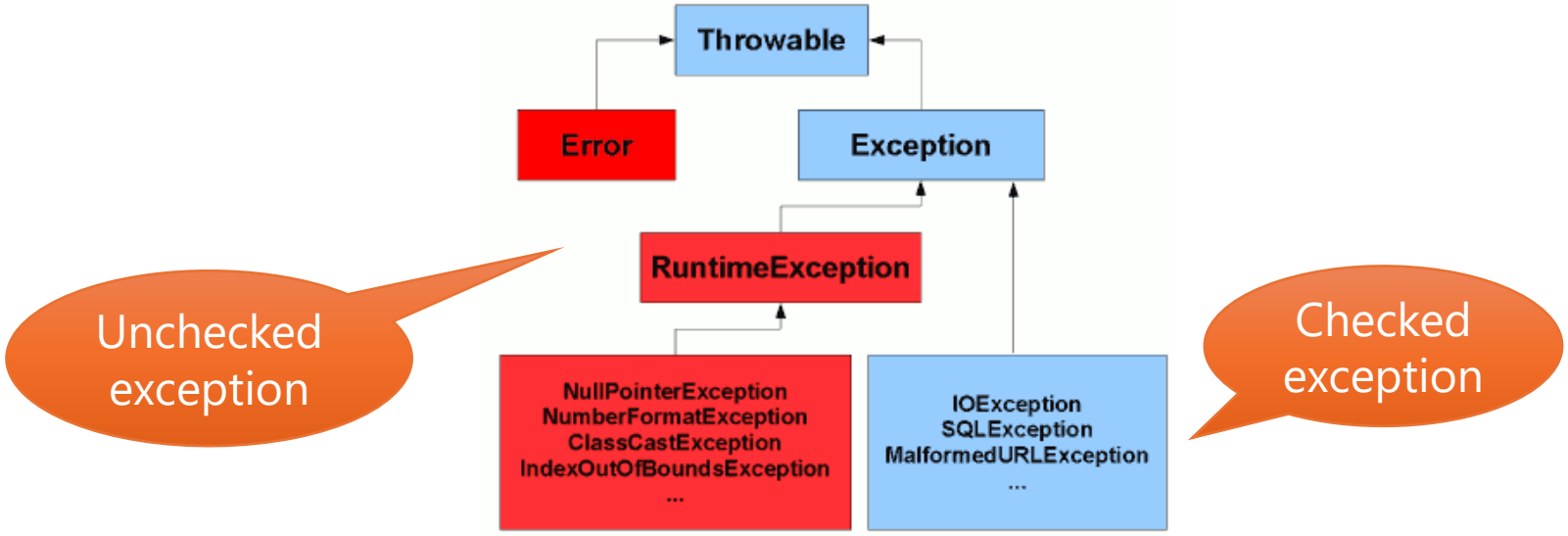
Transaction

- In Spring Data JPA, transaction management ensures atomicity and data consistency by automatically wrapping database operations into a single unit of work.
- If any operation fails, the entire unit rolls back.
- https://docs.spring.io/spring-data/jpa/reference/jpa/transactions.html



@Transactional

- Basic syntax: @Transactional on a method or class.
- @Transactional should be placed in the Service layer (recommended), NOT in the Controller.
- Key attributes: readOnly, rollbackFor, timeout, propagation, isolation
- By default, Spring rolls back upon encountering a **RuntimeException** or **Error**, but does not roll back for a checked exception.



Custom exception

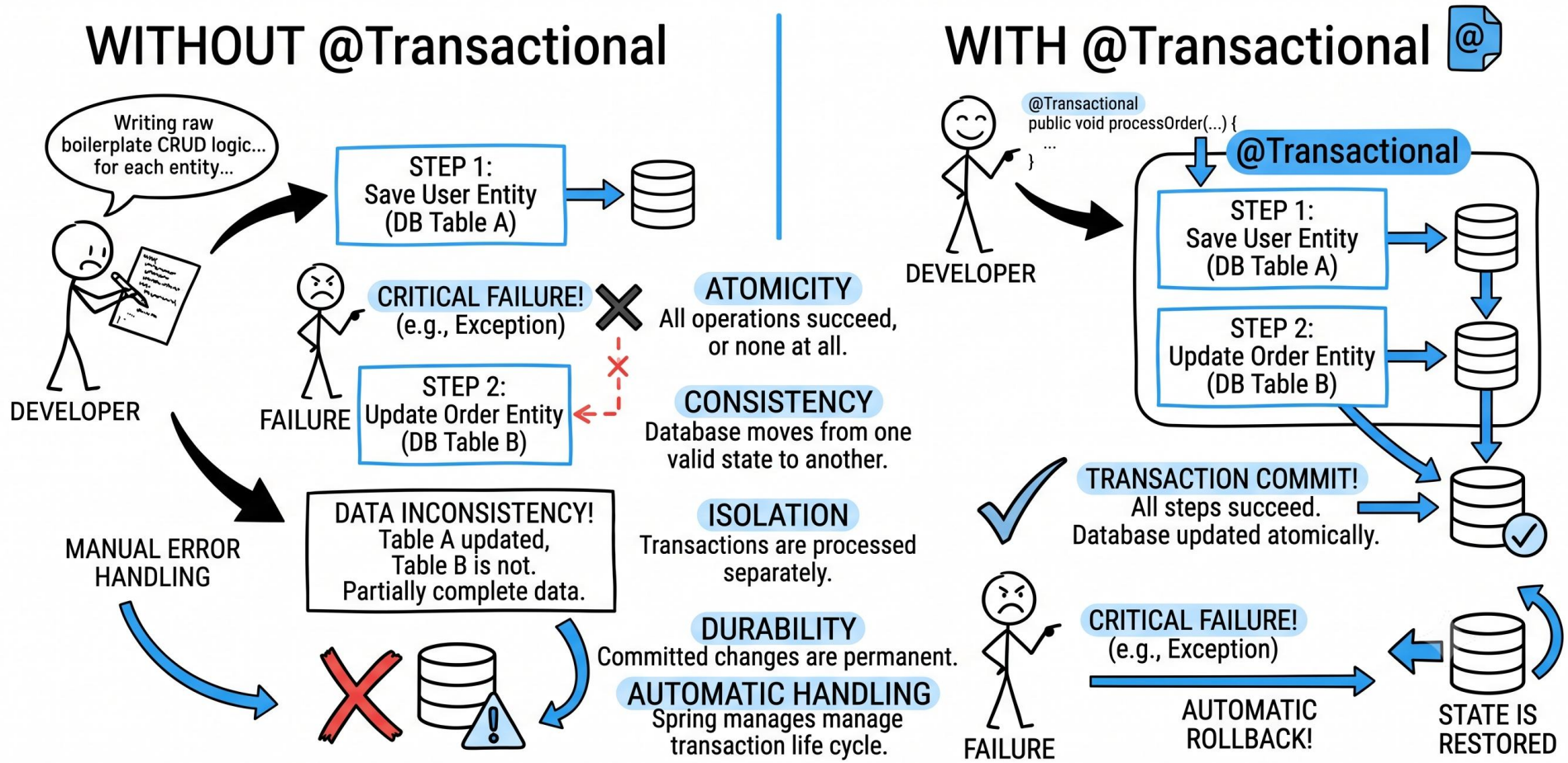
```
public class BookNotFoundException extends RuntimeException {
    public BookNotFoundException(String message) {
        super(message);
    }
}
```

✅ Rollback

```
public class BookNotFoundException extends Exception {
    public BookNotFoundException(String message) {
        super(message);
    }
}
```

❌ NO rollback

Transaction



Q&A – Thank you